Analysis and Design of Algorithms

NUS National University of Singapore

CS3230

Recitation

Week 8

Prof Lee Wee Sun

Worst Case Analysis

Runtime for the worst case input.

- Provides guarantee that holds **at all times**
- Average-case/Expected runtime ≤ Worst case runtime

- Worst case runtime analysis is sometimes easier to do

Probabilistic Analysis, Expected Runtime

For average-case runtime of algorithm:
Must know input distribution $\Pr(i)$.

For randomized algorithms, can sometimes compute expected runtime without knowing input distribution, when expected runtime is independent of input, eg. Randomized quicksort/quickselect.

Amortized Analysis

Analyse **a sequence of operations**, instead of one operation.

- Average cost over the sequence is amortized cost – aggregrate method: find total cost, divide by number of operations.
- May give better bound than assuming worst case cost for each operation.
- A type of worst case analysis (assume worst sequence), not probabilistic analysis (no knowledge of input distr. required)
- More accurate runtime for iterative algorithms

Accounting Method

Assign an amortized cost to each operation such that

- Part of it is used to pay for the cost of the operation
- Unused part is saved in the bank
- Bank balance must never go negative

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i$$

i.e. total amortized cost upper bounds the total true cost.

Potential Method

Define potential function $\Phi(D)$ on the data structure.

- Analogous to bank balance in accounting method
- Non-negative and $\Phi(D_0) = 0$.
- Amortized cost computed as

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = c_i + \Delta\Phi_i$$

- Then have $\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i$

Admin

- PA2: deadline Sunday 23 October midnight (2 week's time). Note that for task B, you can submit to course at most 3 times but have unlimited submission on practice.
- HW3: deadline Friday 28 Oct midnight (3 week's time). Q1 on amortized analysis, Q2 on dynamic programming, Q3 on greedy algorithms.
- PA3: deadline Sunday 6 November midnight (4 week's time). Only one problem.
- Short survey to get feedback (Survey on sidebar on Coursemology): 100 EXP

MCQ

Question 1

True or False: The worst case insert cost for dynamic table is $\Theta(n)$. Hence the worst case runtime for n inserts is $\Theta(n^2)$.

False: For n inserts, the cost of the n inserts (without table doubling) is O(n). The cost of table doubling is no more than

$$1 + 2 + \cdots + 2^{\lfloor \lg n \rfloor} = \Theta(n)$$

So worst case total runtime is O(n).

Or

Amortized cost for insert is O(1). Hence worst case total cost for n inserts is O(n).

Question 2

True or False: Consider a queue with two operations: *inserting* one element and *emptying* the queue, implemented by deleting all the elements one by one. Both the *insert* and *empty* operations are known to take O(1) **amortized** time. This data structure is suitable for applications where every operation must run in a **fixed amount of time** regardless of the number of elements in the queue.

False: Amortized O(1) time does not imply worst case O(1) time. Emptying the queue is Ω (n) time as n elements deleted one by one.

Question 3

When we say that an operation is amortized $\Theta(1)$, we mean that:

- n operations run in total $\Theta(n)$ time in the worst case.
- n operations run in total $\Theta(n)$ time in the expected case.
- n operations run in total $\Theta(n)$ time in the best case.
- n operations run in total $\Theta(1)$ in the expected case.

Solution

Answer: A

Amortized cost of $\Theta(1)$ means that the total cost divided by the number of operations is $\Theta(1)$. Hence the total cost is $\Theta(n)$.

Amortized analysis is done in the worst case.

Question 4

We have a queue-based data structure with the following three operations:

ENQUEUE: Adds an element to the back of the queue.

DEQUEUE: Removes an element from the back of the queue.

CLEAR: Removes elements one at a time from the back of the queue until the queue is empty.

Enqueuing and Dequeuing elements at the back of the list runs in $O(1)$ time. Which of the following are true?

I. CLEAR runs in $\Theta(n)$.

II. The amortized time for each operation is $\Theta(n)$.

III. CLEAR runs in $\Theta(1)$.

IV. The amortized time for each operation is $\Theta(1)$.

- I, II
- II, III
- III, IV
- I, IV

Solution

Answer: D

Clear removes one element at a time so clearly runs in $\Theta(n)$ time.

All operations also run in $\Theta(1)$ amortized time.

If we charge \$2 for enqueue (use \$1 for the operation and keep \$1 with the element), we have \$1 left on each element to use when dequeue or clear. So can charge \$0 for dequeue and clear.

Alternatively, use potential method with number of elements in queue as potential.

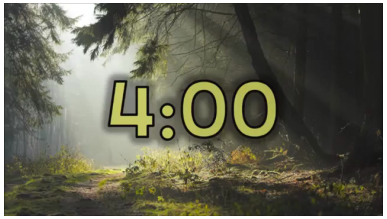
- *Enqueue*: Actual cost is 1 and change of potential is 1, giving amortized cost of 2.
- *Dequeue*: Actual cost is 1 and change of potential is -1, giving amortized cost of 0.
- *Clear*: Actual cost is k (size of queue) and change in potential is $-k$, giving amortized cost of 0.

Question 5

The amortized time for EXTRACT-MIN is $O(\lg V)$ while the amortized time for DECREASE-KEY is $O(1)$ for a Fibonacci heap. Consider the pseudocode for Prim's algorithm for finding the minimum spanning tree.

```
Q ← V

key[v] ← ∞

for all v ∈ V

    key[s] ← 0 for some arbitrary s ∈ V

while Q ≠ ∅

    do u ← EXTRACT-MIN(Q)

        for each v ∈ Adj[u]

            do if v ∈ Q and w(u, v) < key[v]

                then key[v] ← w(u, v)

                    π[v] ← u
```

- $O(V \log V + E \log V)$
- $O(V \log V + V^2 \log V)$
- $O(E + V \log V)$



Which of the following gives the tightest bound on the running time of Prim's algorithm when the priority queue is implemented using Fibonacci heap?

Solution

Answer: C

V extract-min operation and E decrease-key.

Extract-min has amortized cost of $O(\log V)$ while decrease-key has amortized cost of $O(1)$.

So total cost of $O(V \log V + E)$.

Question 6

Who is the Master of Algorithms pictured below?

- Daniel Sleator
- Ron Rivest
- John Hopcroft
- Robert Tarjan

Question 6

Robert Tarjan

Turing Award winner

- Linear time select (with Floyd and 3 others), least common ancestor, connected components, Fibonacci heap, splay trees.
- Many of the data structures analysed with amortized analysis.

Structured Questions

Question 1

Queue with Find-Max

We want to implement a queue that can run the following operations in amortised $O(1)$:

- ENQUEUE
- DEQUEUE
- FIND-MAX

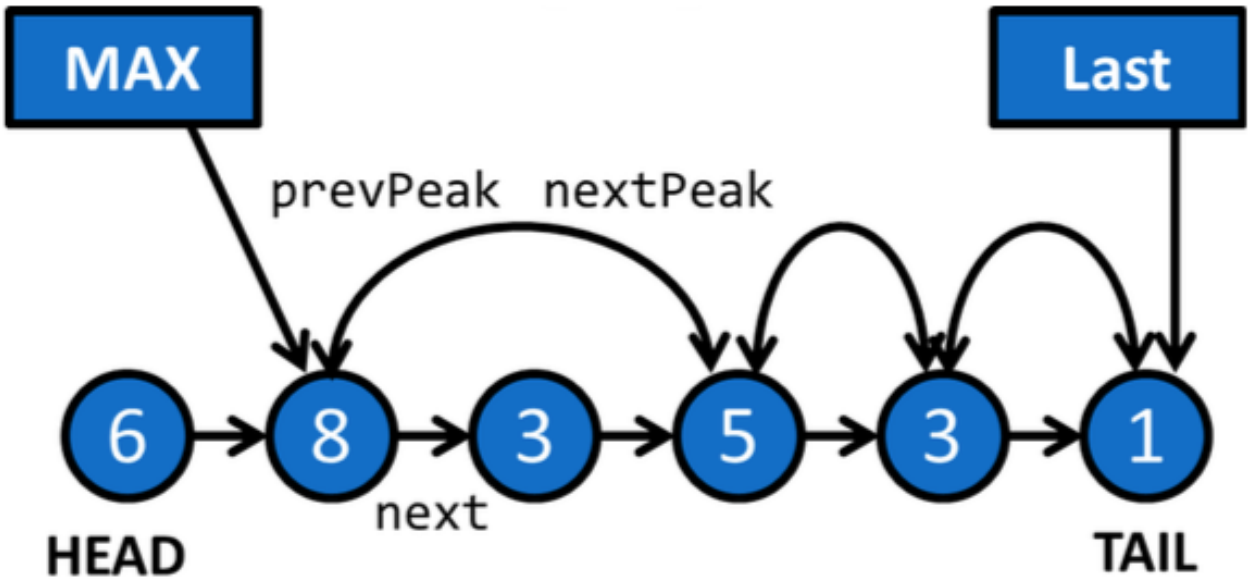
We make the following definition:

In a sequence of numbers, a **Peak** is a value in the sequence that is larger than all the values in the sequence after it.

For example, in the sequence $[3, 7, 5, 1, 6, 4, 6, 3, 1, 2]$, the peaks are $[7, 6, 3, 2]$. Consequently, the first peak in the sequence is the largest element in the sequence.

The queue is implemented as a standard linkedlist-based queue. We have an additional pointer, MAX, which points to the largest element (the first peak) in the queue. Each node in the queue may have up to two additional pointers nextPeak and prevPeak, which point to the next and previous peaks respectively.

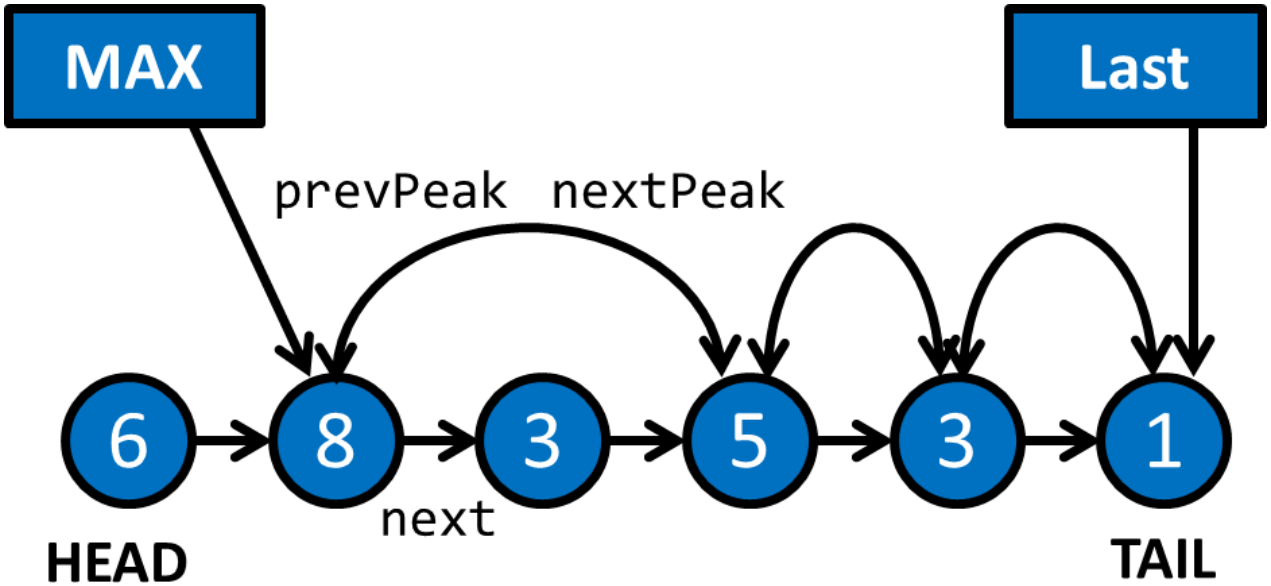
This is illustrated in the following diagram:



In this case, the peaks are $[8, 5, 3, 1]$.

We have pointers to both the head and tail of the queue for enqueueing and dequeueing. Assume that we enqueue to the right, and dequeue from the left.

Question 1(a)



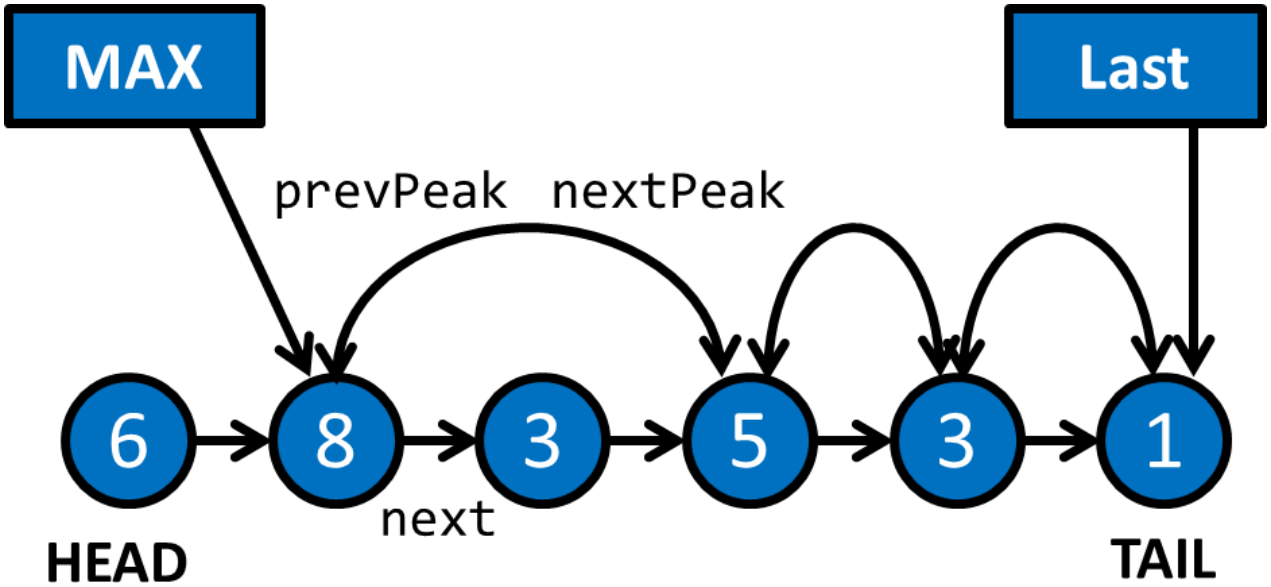
Describe how the DEQUEUE operation can be implemented so that the above structure is maintained after every operation.
[Hint: use the nextPeak pointer to go to the next peak]

Solution: We simply remove the first element.
If the first element is also the MAX, we reassign MAX to the next peak.

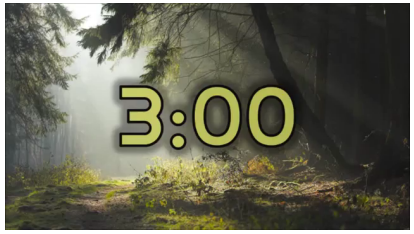
Pseudocode:

```
if list.first() = MAX
        MAX ← MAX.nextPeak
list.removeFirst()
```

Question 1(b)



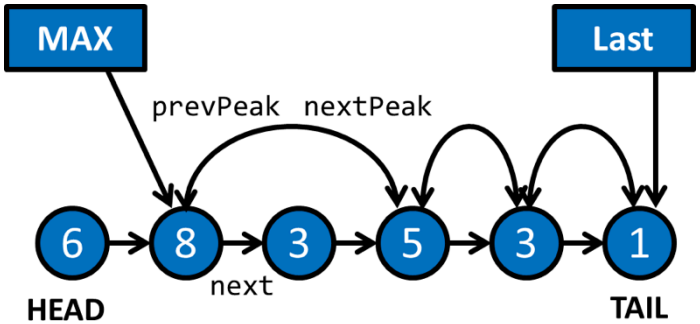
Describe how the ENQUEUE operation can be implemented so that the above structure is maintained after every operation.
[Hint: use the prevPeak pointer to go to the previous peak]



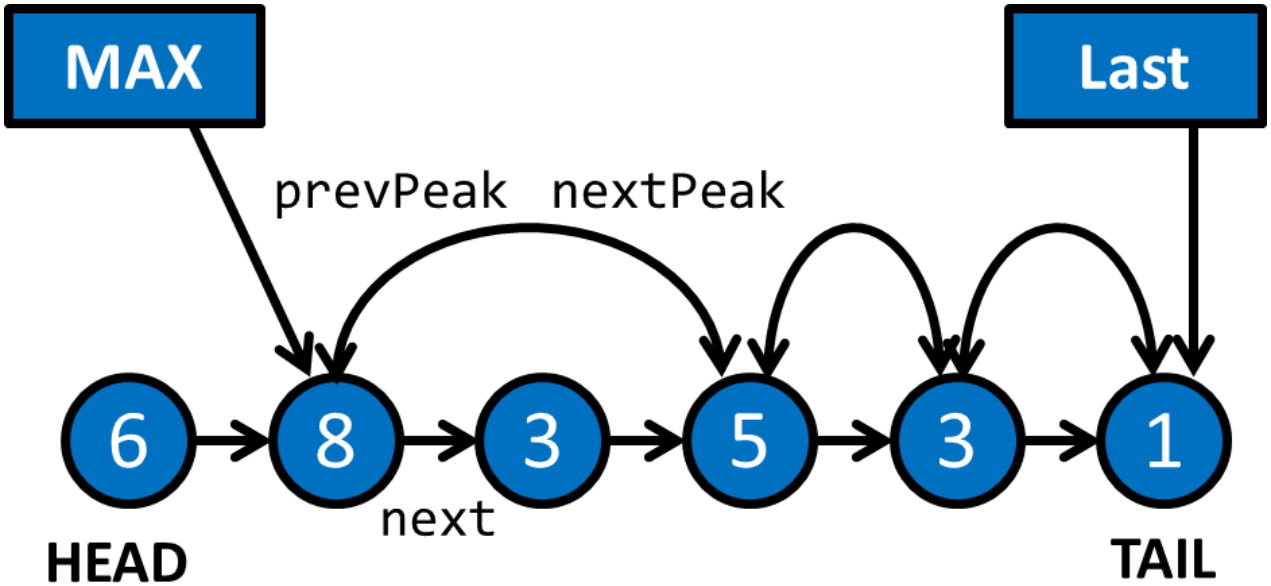
Solution: Suppose we want to add the value V.
Starting from the last peak (which is the last node in the list), we traverse backward using the prevPeak pointers until we find a peak larger than V. We then connect this peak to the newly added node via the nextPeak/prevPeak pointers.
If all peaks are $\leq V$, we set the MAX pointer to the newly added node.

ENQUEUE

1. Create a node N with value V
2. current ← list.last()
3. **while** current != null,
4. **if** current.value > V,
5. current.nextPeak ← N
6. N.prevPeak ← current
7. break
8. current ← current.prevPeak
9. **if** current = null,
10. MAX ← N
11. Insert N at the end of list.



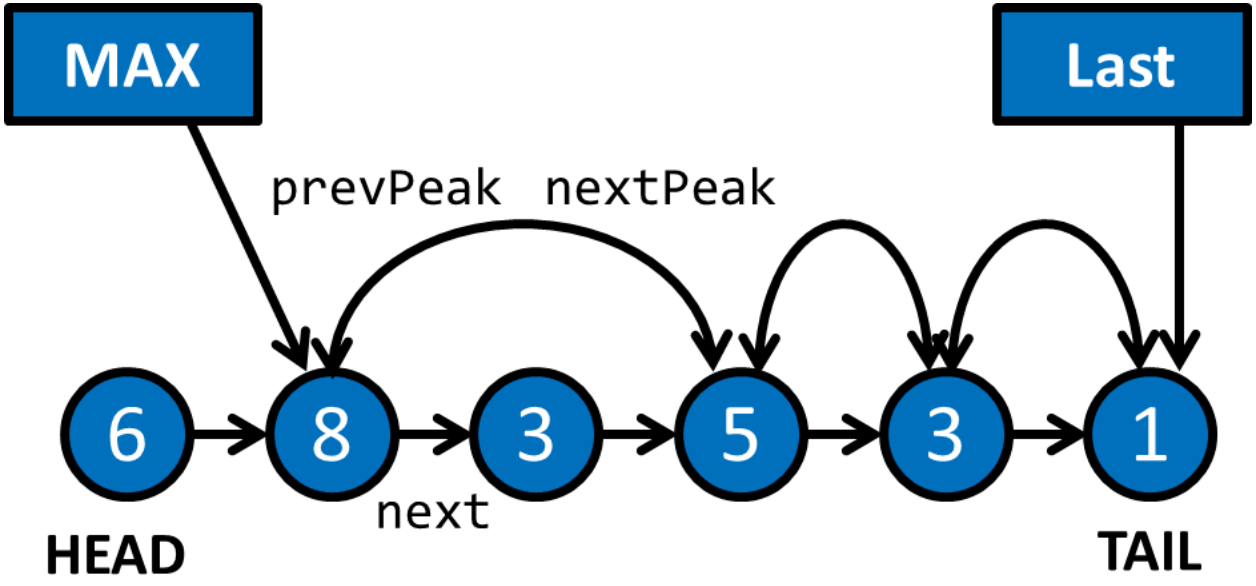
Question 1(c)



Describe how the FIND-MAX operation is implemented.

FIND-MAX

1. return MAX.value



Question 1(d)

Give amortized costs for the ENQUEUE, DEQUEUE, and FIND-MAX operations. Prove that the bank balance never goes negative after any ENQUEUE, DEQUEUE, or FIND-MAX operation. Hence conclude that these three operations run in amortised $O(1)$.

1. Enqueue: $2
2. Dequeue: $1
3. Find-Max: $1

Invariant before each operation:
There is $1 stored at every peak in the list.
(therefore bank account never negative)

Initially true with empty list

1. Enqueue: $2
2. Dequeue: $1
3. Find-Max: $1

Invariant before each operation:

There is $1 stored at every peak in the list.

Dequeue:

- $1 for constant amount of work.
- Discard $1 if removing a peak maintaining invariant.

1. Enqueue: $2
2. Dequeue: $1
3. Find-Max: $1

Invariant before each operation:

There is $1 stored at every peak in the list.

Find-Max:

- $1 for constant amount of work.
- Nothing else happens, maintaining invariant.

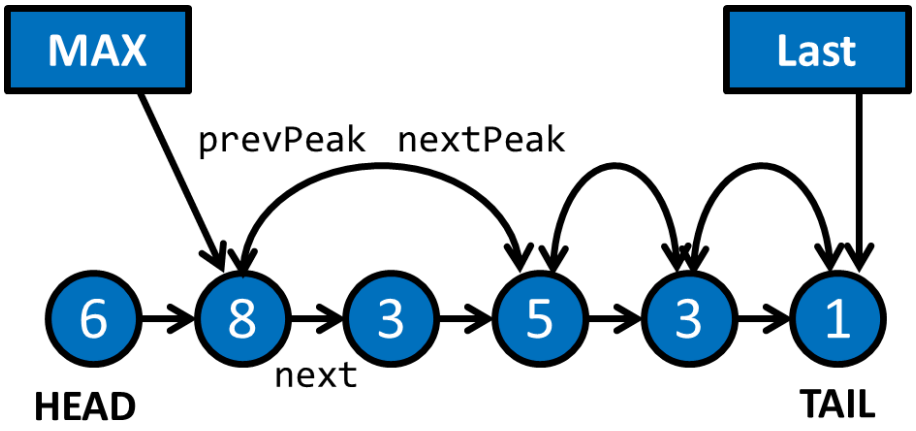
1. Enqueue: $2
2. Dequeue: $1
3. Find-Max: $1

Invariant before each operation:

There is $1 stored at every peak in the list.

Enqueue:

- Add a node at end of queue and store $1 with the node.



Enqueue: $2

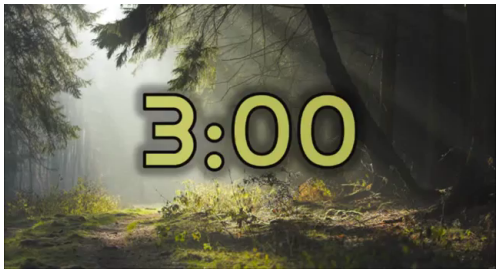
Invariant before each operation:

There is $1 stored at every peak in the list.

- Loop using prevPeak until find peak larger than new node: use $1 stored at each peak to remove the peak within loop
- Additional $1 pays for creating node, inserting node, and work when loop terminated ($1 inserted at new node, $1 for constant amount of other work, total $2)

Question 1(e)

You work for an online stock broker. You are asked to write a program to display the maximum price for each stock over the past year, and to update the display each day. More generally, consider the case where you are asked to compute the maximum price over the past k days. Design an efficient algorithm to do the task. What is the total runtime of your algorithm for doing the computation for a single stock over n days. What is the worst case runtime for any one day?



Solution: Use the queue with find-max data structure for each stock. At the start, insert the stock price for the current day until k days has been reached. Then for each day, remove a stock price (the oldest price) and insert a new one. As the amortized cost for enqueue, dequeue, and find-max are all $O(1)$, the total runtime for n days is $O(n)$, independent of the value of k. The worst case runtime for a single day is $\Omega(k)$ from the worst case runtime for enqueue. Dequeue and find-max both takes constant time in the worst case.